

Модуль 10. Apache Kafka: Topics, Partitions, Offsets и Consumer Groups

10.1. Зачем Kafka: от комнатной библиотеки к государственному архиву

В Модуле 9 мы поняли, что мир делится на **команды** (Celery) и **факты** (Kafka). Redis Streams дал нам первый вкус персистентного лога, но Redis — это, по сути, умная оперативная память. Он хранит данные в RAM, и если вам нужно накопить терабайты событий за год — он либо не подойдёт, либо обойдётся в астрономическую сумму.

Apache Kafka — это промышленная распределённая система, спроектированная именно для логов событий. Она была создана в LinkedIn для обработки миллиардов сообщений в день, и сегодня используется практически в любой крупной компании.

Аналогия: от дневника к Национальному архиву

- **Celery + Redis** — это ваш личный ежедневник. Вы пишете: «Сегодня позвонить Васе». Запись эфемерна, страницы вырываются по мере выполнения.
- **Redis Streams** — это дневник, который вы не вырываете. Можно перечитать, но записная книжка толстеет, и через месяц она развалится от объёма.
- **Kafka** — это **государственный архив**. Огромное здание с бесконечными полками. Каждый документ лежит в строго определённом месте, пронумерован, защищён от пожара (репликация), и к нему могут одновременно обращаться сотни исследователей, не мешая друг другу.

В этом модуле мы разберём, из чего состоит этот архив.

10.2. Архитектура Kafka: пять китов

Прежде чем запускать команды, нужно понять пять фундаментальных понятий. Без них команды будут казаться магией.

10.2.1. Topic (Топик): название газеты

Топик — это **категория** или **канал** событий. Это просто строка-имя, под которой группируются родственные сообщения.

Аналогия: В киоске газет лежат издания: «Спорт», «Политика», «Наука». Каждое издание — это топик. Издательство (Producer) выпускает новый номер «Спорта» — и он попадает именно в стопку «Спорт», а не в «Политику».

Примеры топиков в ML-системе:

- `user.clicks` — клики пользователей.

- `model.predictions` — факты выполнения предсказаний.
- `dataset.uploads` — загрузки датасетов.
- `orders.completed` — завершённые заказы.

Важно: Топик — это логическая сущность. Физически данные хранятся иначе (в партициях).

10.2.2. Partition (Партиция): тома энциклопедии

Если бы вся газета «Спорт» печаталась на одной ленте конвейера, она быстро стала бы узким местом. Поэтому издательство разбивает поток на **несколько параллельных лент**.

Партиция — это **физический сегмент** топика. Каждый топик разбит на одну или несколько партиций. Каждая партиция — это **упорядоченный, неизменяемый лог** сообщений.

Аналогия: многотомная энциклопедия

Представьте, что вы издаёте «Энциклопедию событий». Она не помещается в один том. Вы делите её на тома:

- Том 1 (Партиция 0): события с 1 января по 30 июня.
- Том 2 (Партиция 1): события с 1 июля по 31 декабря.
- Том 3 (Партиция 2): дополнительные материалы.

Каждый том — это отдельная книга со своей нумерацией страниц. События внутри тома идут строго по порядку. Но между томами порядок не гарантирован (событие в томе 2 могло произойти раньше события в томе 1, если разделение было не по времени, а по ключу).

Зачем нужны партиции:

1. **Масштабирование записи.** Producer может писать в разные партиции параллельно.
2. **Масштабирование чтения.** Разные Consumer'ы могут читать разные партиции параллельно.
3. **Объём.** Одна партиция — это файлы на диске. Разбивая топик, мы разбиваем нагрузку по дискам и серверам.

Как сообщение попадает в партицию?

Если Producer не указал **ключ** (key), сообщения распределяются **round-robin** (по кругу): первое в партицию 0, второе в 1, третье в 2, четвёртое снова в 0...

Если Producer указал ключ (например, `key="user_42"`), Kafka вычисляет хеш ключа и всегда отправляет сообщения с этим ключом в **одну и ту же партицию**. Это критично, когда нужно сохранить порядок событий одного пользователя.

10.2.3. Offset (Оффсет): номер страницы

Offset — это **порядковый номер** сообщения внутри партиции. Начинается с 0 и увеличивается на 1 с каждым новым сообщением.

Аналогия: Номер страницы в книге. Вы читаете том 1 (партицию 0) и остановились на странице 152 (offset 152). Завтра вы приходите, открываете тот же том на закладке 152 и продолжаете. Вам не нужно перечитывать с начала.

Важнейшее свойство: Offset назначается **партицией** при записи сообщения. Consumer не может его изменить. Это не timestamp (хотя timestamp тоже есть), это строгая числовая последовательность.

Offset — это позиция Consumer'a. Каждая Consumer Group запоминает, до какого offset'a она дочитала каждую партицию. Это позволяет:

- Перезапустить Consumer — он продолжит с места остановки.
- Добавить нового Consumer'a в группу — Kafka перераспределит партиции.
- «Перемотать» назад и перечитать старые сообщения.

10.2.4. Broker (Брокер): библиотекарь и хранилище

Broker — это сервер Kafka, который хранит данные и обслуживает запросы. В production Kafka работает в кластере из 3, 5, 7 брокеров.

Аналогия: Библиотекарь в огромном архиве. Он знает, в каком зале (на каком сервере) лежит том 1, а в каком — том 2. Он выдаёт книги читателям и принимает новые от издателей.

В нашем учебном примере мы запустим **один** брокер в Docker. Но даже он уже полноценный Kafka-сервер.

10.2.5. Consumer Group (Группа потребителей): команда читателей

Consumer Group — это логическая группа из одного или нескольких Consumer'ов, которые **совместно читают один топик**.

Аналогия: Группа студентов пишет курсовую по одной и той же многотомной энциклопедии. Они договариваются:

- Петя читает том 1 (партицию 0).
- Вася читает том 2 (партицию 1).
- Маша читает том 3 (партицию 2).

Каждый ведёт свои заметки (свои offset'ы). Они не мешают друг другу. Книги остаются на полках (сообщения не удаляются).

Правило распределения:

- **Одна партиция может быть назначена только одному Consumer'у внутри группы.** (Если Петя читает том 1, Вася не может его читать в той же группе.)
- **Один Consumer может читать несколько партиций.** (Если в группе только Вася — он читает все три тома.)
- **Если Consumer'ов больше, чем партиций — лишние простаивают.** (Если студентов 5, а томов 3 — двое бездельничают.)

Зачем группы?

- **Параллелизм.** Три Consumer'а в группе читают в 3 раза быстрее (если партиций ≥ 3).
- **Отказоустойчивость.** Если Петя заболел (его Consumer упал), Kafka автоматически переназначает его партицию Васе или Маше (**ребалансировка**).

10.3. Репликация и отказоустойчивость: лидер и последователи

В production Kafka никогда не хранит данные в одном экземпляре. Каждая партиция имеет **реплики** (копии) на разных брокерах.

Replication Factor (коэффициент репликации) = сколько копий существует. Обычно 3.

Роли:

- **Leader (лидер):** Основная реплика. В неё пишет Producer. Из неё читает Consumer (обычно).
- **Follower (последователь, реплика):** Пассивная копия. Она постоянно «подглядывает» в Leader и копирует новые сообщения. Не обслуживает клиентов напрямую.

Что происходит при сбое:

1. Брокер с Leader падает.
2. Kafka выбирает одного из Follower'ов и делает его новым Leader'ом.
3. Producer и Consumer автоматически переключаются на нового Leader'а.
4. Человек (разработчик) об этом даже не узнает, если всё настроено правильно.

В нашем учебном примере мы используем `replication-factor=1` (нет реплик), потому что у нас один брокер в Docker. В production всегда ≥ 3 .

10.4. KRaft vs ZooKeeper: кто управляет зоопарком?

Раньше Kafka для координации кластера требовала **ZooKeeper** — отдельную распределённую систему. Это было сложно: нужно было поднимать и Kafka, и ZooKeeper, и следить за их взаимодействием.

Начиная с Kafka 3.3+ появился режим **KRaft** (Kafka Raft). В нём Kafka управляет собой сама, без ZooKeeper. Для обучения это проще.

Мы будем использовать KRaft (без ZooKeeper), потому что:

- Меньше контейнеров.
- Современный подход.
- Для одного брокера это идеально.

10.5. Практика: разворачиваем Kafka в Docker

Шаг 1. Создаём рабочую папку и docker-compose.yml

```
In [1]: mkdir ~/docker-module10
        cd ~/docker-module10
```

Создайте файл `docker-compose.yml` :

```
In [2]: services:
        kafka:
          image: docker.io/bitnami/kafka:3.7
          container_name: kafka_broker
          ports:
            # Порт для подключения Producer'ов и Consumer'ов
            - "9092:9092"
          environment:
            # Уникальный ID брокера в кластере
            - KAFKA_CFG_NODE_ID=0

            # Этот контейнер будет и брокером, и контроллером (KRaft)
            - KAFKA_CFG_PROCESS_ROLES=controller,broker

            # Сетевые интерфейсы:
            # PLAINTEXT — для клиентов (наш хост и контейнеры)
            # CONTROLLER — для внутреннего управления KRaft
            - KAFKA_CFG_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093
```

```
- KAFKA_CFG_ADVERTISED_LISTENERS=PLAINTEXT://localhost:9092
- KAFKA_CFG_CONTROLLER_LISTENER_NAMES=CONTROLLER
- KAFKA_CFG_LISTENER_SECURITY_PROTOCOL_MAP=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT

# Кворум: голосование за лидера. У нас 1 брокер, он сам себе контроллер
- KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=0@kafka:9093

# Автоматически создавать топики при первом обращении (удобно для обучения)
- KAFKA_CFG_AUTO_CREATE_TOPICS_ENABLE=true

# По умолчанию создавать 3 партии у новых топиков
- KAFKA_CFG_NUM_PARTITIONS=3
```

Разбор ключевых переменных простым языком:

- `KAFKA_CFG_NODE_ID=0` — имя брокера. В кластере из 5 серверов у каждого был бы свой ID (0, 1, 2, 3, 4).
- `PROCESS_ROLES=controller,broker` — этот сервер выполняет две роли: хранит данные (broker) и управляет кластером (controller). В большом кластере это разные серверы.
- `ADVERTISED_LISTENERS` — адрес, который Kafka сообщает клиентам. Мы говорим: «Подключайтесь по localhost:9092».
- `AUTO_CREATE_TOPICS_ENABLE=true` — если Producer пишет в несуществующий топик, Kafka создаст его автоматически. В production обычно отключают, но для обучения удобно.

Запустите:

```
In [3]: docker compose up -d
```

Проверьте, что Kafka жив:

```
In [4]: docker logs kafka-broker | grep "Kafka Server started"
```

Вы должны увидеть сообщение о успешном старте.

10.6. Работа с Kafka через консольные утилиты

Bitnami-образ содержит все консольные скрипты внутри контейнера. Мы будем запускать их через `docker exec`.

10.6.1. Создание топика вручную

Хотя у нас включено авто-создание, научимся создавать топик явно:

```
In [5]: docker exec -it kafka_broker \  
        kafka-topics.sh \  
        --create \  
        --topic user.events \  
        --bootstrap-server localhost:9092 \  
        --partitions 3 \  
        --replication-factor 1
```

Разбор команды:

- `kafka-topics.sh` — утилита управления топиками.
- `--create` — создать.
- `--topic user.events` — имя.
- `--bootstrap-server localhost:9092` — адрес Kafka.
- `--partitions 3` — разбить на 3 партии.
- `--replication-factor 1` — одна копия (потому что один брокер).

Результат:

```
In [6]: Created topic user.events.
```

Посмотреть список топиков:

```
In [7]: docker exec -it kafka_broker \  
        kafka-topics.sh \  
        --list \  
        --bootstrap-server localhost:9092
```

Посмотреть детали топика:

```
In [8]: docker exec -it kafka_broker \  
        kafka-topics.sh \  
        --describe \  
        --topic user.events
```

```
--topic user.events \  
--bootstrap-server localhost:9092
```

Вы увидите что-то вроде:

```
In [9]: Topic: user.events      TopicId: aBcD...      PartitionCount: 3      Replicatio  
        nFactor: 1  
        Partition: 0      Leader: 0      Replicas: 0      Isr: 0  
        Partition: 1      Leader: 0      Replicas: 0      Isr: 0  
        Partition: 2      Leader: 0      Replicas: 0      Isr: 0
```

Это означает: топик разбит на 3 партии, все они живут на брокере 0.

10.6.2. Отправка сообщений (Producer)

Запустите интерактивного Producer'a:

```
In [10]: docker exec -it kafka_broker \  
        kafka-console-producer.sh \  
        --topic user.events \  
        --bootstrap-server localhost:9092
```

Терминал зависнет в ожидании ввода. Печатайте сообщения и жмите Enter:

```
In [11]: {"user_id": 1, "action": "login", "timestamp": "2026-08-20T10:00:00Z"}  
{"user_id": 2, "action": "click", "button": "buy", "timestamp": "2026-08-20T10:0  
1:00Z"}  
{"user_id": 1, "action": "logout", "timestamp": "2026-08-20T10:05:00Z"}
```

Каждая строка — это одно сообщение. Kafka не знает про JSON, для него это просто текст.

Что происходит под капотом: Producer отправляет сообщение брокеру. Брокер выбирает партицию (round-robin, так как нет ключа) и записывает сообщение в конец лога этой партиции.

10.6.3. Чтение сообщений (Standalone Consumer)

Откройте **второй** терминал и запустите Consumer'a:


```
In [12]: docker exec -it kafka_broker \
         kafka-console-consumer.sh \
         --topic user.events \
         --from-beginning \
         --bootstrap-server localhost:9092
```

Разбор:

- `--from-beginning` — читать с самого начала (с offset 0). Без этого флага Consumer читал бы только новые сообщения, появившиеся после его запуска.
- `--bootstrap-server` — адрес Kafka.

Вы увидите все три сообщения, которые отправили ранее. Они не исчезли из топика!

Важное наблюдение: Сообщения остались в Kafka. Вы можете остановить Consumer (`Ctrl+C`), запустить снова — и снова увидите те же сообщения (если снова используете `--from-beginning`).

10.6.4. Чтение в Consumer Group

Теперь проверим группы. Запустите **трёх** Consumer'ов в **одной** группе в **трёх разных** терминалах.

Терминал 1:

```
In [13]: docker exec -it kafka_broker \
         kafka-console-consumer.sh \
         --topic user.events \
         --group ml_analytics \
         --bootstrap-server localhost:9092
```

Терминал 2:

```
In [14]: docker exec -it kafka_broker \
         kafka-console-consumer.sh \
         --topic user.events \
         --group ml_analytics \
         --bootstrap-server localhost:9092
```

Терминал 3:

```
In [15]: docker exec -it kafka_broker \
        kafka-console-consumer.sh \
        --topic user.events \
        --group ml_analytics \
        --bootstrap-server localhost:9092
```

Теперь отправьте **новые** сообщения через Producer (в первом терминале):

```
In [16]: {"event": "new_1"}
        {"event": "new_2"}
        {"event": "new_3"}
        {"event": "new_4"}
        {"event": "new_5"}
        {"event": "new_6"}
```

Наблюдайте: Kafka распределила 3 партии между 3 Consumer'ами. Каждый Consumer получает сообщения только из «своих» партиций. Вы увидите, что сообщения разошлись по разным терминалам.

Проверьте назначение:

```
In [17]: docker exec -it kafka_broker \
        kafka-consumer-groups.sh \
        --bootstrap-server localhost:9092 \
        --describe \
        --group ml_analytics
```

Вы увидите таблицу:

```
In [18]: GROUP          TOPIC          PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
CONSUMER-ID
ml_analytics consumer-1      0          2              2              0
ml_analytics consumer-2      1          2              2              0
ml_analytics consumer-3      2          2              2              0
```

Разбор столбцов:

- `PARTITION` — номер партиции.
- `CURRENT-OFFSET` — до куда дочитал группа.
- `LOG-END-OFFSET` — сколько всего сообщений в партиции.
- `LAG` — сколько сообщений не прочитано (0 = всё прочитано).
- `CONSUMER-ID` — какой конкретно Consumer читает эту партицию.

10.6.5. Другая Consumer Group читает независимо

Запустите Consumer в **другой** группе:

```
In [19]: docker exec -it kafka_broker \
         kafka-console-consumer.sh \
         --topic user.events \
         --group audit_service \
         --from-beginning \
         --bootstrap-server localhost:9092
```

Он прочитает **все** сообщения с начала, независимо от `ml_analytics`. Потому что offset'ы хранятся **по группам**. `audit_service` ведёт свои собственные закладки.

10.7. Эксперимент: ребалансировка группы

Цель: Увидеть, как Kafka перераспределяет партиции, когда Consumer выходит из строя.

1. Убедитесь, что три Consumer'а в группе `ml_analytics` работают.
2. В одном из терминалов нажмите `Ctrl+C` (Consumer ушёл).
3. Отправьте новые сообщения через Producer.

Наблюдайте: Оставшиеся два Consumer'а продолжат читать. Один из них «подхватит» партицию упавшего коллеги. Это и есть **ребалансировка** (rebalance).

Проверьте:

```
In [20]: docker exec -it kafka_broker \
         kafka-consumer-groups.sh \
         --bootstrap-server localhost:9092 \
         --describe \
         --group ml_analytics
```

Теперь одному Consumer'у назначено 2 партиции, другому — 1.

10.8. Что происходит на диске: файлы лога

Kafka хранит каждую партицию как **папку с файлами** на диске. Заглянем внутрь контейнера:

```
In [21]: docker exec -it kafka_broker ls /bitnami/kafka/data/
```

Вы увидите папки вида:

```
In [22]: user.events-0
user.events-1
user.events-2
```

Каждая папка — это партиция. Внутри:

```
In [23]: docker exec -it kafka_broker ls /bitnami/kafka/data/user.events-0
```

Там лежат файлы `.log`, `.index`, `.timeindex`. Это и есть физический лог сообщений. Kafka пишет на диск **последовательно** (append-only), что делает запись невероятно быстрой, даже на обычных HDD.

10.9. Итоги модуля: чек-лист

- ☐ Понимаю, что **Topic** — это категория/канал событий (как газета).
- ☐ Знаю, что **Partition** — это физический сегмент топика, упорядоченный лог.
- ☐ Понимаю, зачем нужно несколько партиций: параллелизм записи и чтения.
- ☐ Знаю, что сообщения внутри одной партиции упорядочены, а между партициями — нет.
- ☐ Понимаю, что **Offset** — это номер позиции внутри партиции (как страница в книге).
- ☐ Знаю, что offset'ы хранятся **по Consumer Group**, и каждая группа читает независимо.
- ☐ Понимаю, что **Consumer Group** делит партиции между участниками: одна партиция — один consumer в группе.
- ☐ Знаю, что если consumer'ов больше, чем партиций — лишние простаивают.
- ☐ Понимаю **репликацию**: Leader обрабатывает запросы, Follower'ы копируют данные.
- ☐ Знаю разницу между **ZooKeeper** (старая Kafka) и **KRaft** (современная Kafka без ZooKeeper).
- ☐ Развернул Kafka в Docker с KRaft.
- ☐ Создал топик через `kafka-topics.sh`, проверил его описание.

- ☐ Отправил сообщения через `kafka-console-producer.sh` .
- ☐ Прочитал сообщения через `kafka-console-consumer.sh` с `--from-beginning` .
- ☐ Запустил несколько Consumer'ов в одной группе и увидел распределение партиций.
- ☐ Запустил Consumer в другой группе и убедился в независимом чтении.
- ☐ Увидел **ребалансировку** при выходе Consumer'a из строя.
- ☐ Заглянул в папки с данными Kafka на диске.

В следующем модуле мы научимся работать с Kafka из Python через асинхронный клиент **aiokafka**. Мы интегрируем его в FastAPI: endpoint будет публиковать события, а отдельная корутина-консьюмер будет их обрабатывать.